

Response Helper

In Laravel, We have a `response()` helper that allows you to return a response object. The response helper can be used for many things such as:

- Setting the HTTP response stats
- Setting Header Values
- Response body, which refers to the main content that the server sends back to the client
- Set Cookies
- Set Session data

Up to this point, we have just been returning strings from our route closures. Laravel does some magic behind the scenes to convert these strings into a response object. However, you can also use the `response()` helper if you want more control over the response.

To return a response object, you can use the `response()` helper function like so:

```
Route::get('/test', function () {  
    return response('Hello, World!');  
});
```

When you call `response()`, it internally references the `Illuminate\Http\Response` class and its methods to construct the appropriate response. It is a global helper function, so you do not need to add any `use` statements to use it.

Status Code

You can set the status code of the response by passing it as the second argument to the `response()` function. For example, to return a `404 Not Found` response, you can do this:

```
Route::get('/test', function () {  
    return response('Hello World', 200);  
});
```

So for a `404 Not Found` response, you can do this:

```
Route::get('/test', function () {  
    return response('Not Found', 404);  
});
```

The `response()` helper is syntactic sugar. Without using the helper, you could do the following:

```
use Illuminate\Http\Response;

Route::get('/test', function () {
    return new Response('Not Found', 404);
});
```

Headers

You can set headers on the response by chaining the `header()` method on the response object. For example, to set a `Content-Type` header, you can do this:

```
Route::get('/test', function () {
    return response('<h1>Hello World</h1>')->header('Content-Type', 'text/plain');
});
```

Notice, it is not parsing the HTML tags because we set the `Content-Type` header to `text/plain`. If I change it to the following, it will parse the HTML tags:

```
Route::get('/test', function () {
    return response('<h1>Hello World</h1>')->header('Content-Type', 'text/html');
});
```

It will also parse if I remove the `Content-Type` header because the default is `text/html`.

Responding with JSON

If we wanted to return a JSON response, Laravel actually does this automatically just by returning an array, but we can do it explicitly:

```
Route::get('/test', function () {
    return response()->json(['name' => 'John Doe']);
});
```

Downloading A File

We can download files. The `public_path` helper is used to access the `public` folder, so we could do something like this:

```
Route::get('/download', function () {
    return response()->download(public_path('favicon.ico'));
});
```

This will download the favicon in that folder.

Cookies

You can set cookies on the response by chaining the `cookie()` method on the response object. For example, to set a cookie named `name` with the value `John Doe`, you can do this:

```
Route::get('/test', function () {  
    return response('Hello World')->cookie('name', 'John Doe');  
});
```

If you look in the browser's developer tools, you will see the cookie has been set and the value is encrypted. This is for security. However, when you use it, Laravel will decrypt it for you. Let's add a route to decrypt the cookie value:

```
Route::get('/read-cookie', function (Request $request) {  
    $cookieValue = $request->cookie('hello');  
    return response()->json(['cookie' => $cookieValue]);  
});
```

Now visit <http://localhost:8000/read-cookie> and you will see the cookie value.

You can also deal with session data, stream data and much more. I just wanted to give you a taste of what you can do with the response object. You can check out the [official documentation](#) for more information.